

```
⑥ def draw(self):  
    self.window.blit(self.image, (self.x, self.y))
```

Listing 6-1: The new `Ball` class

When we instantiate a `Ball` object, the `__init__()` method receives three pieces of data: the window to draw into, the width of the window, and the height of the window ❶. We save the `window` variable into the instance variable `self.window` so that we can use it later in the `draw()` method, and we do the same with the `self.windowHeight` and `self.windowWidth` instance variables. We then load the image of the ball using the path to the file and get the `rect` of that ball image ❷. We need the `rect` to calculate the maximum values for `x` and `y` so that the ball will always fully appear in the window. Next, we pick a randomized starting location for the ball ❸. Finally, we set the speed in the `x` and `y` directions to a random value between `-4` and `4` (but not `0`), representing the number of pixels to move per frame ❹. Because of these numbers, the ball may move differently each time we run the program. All these values are saved in instance variables to be used by other methods.

In the main program, we'll call the `update()` method in each frame of the main loop, so this is where we place the code that checks for the ball hitting any border of the window ❺. If it does hit an edge, we reverse the speed in that direction and modify the `x`- and `y`-coordinates (`self.x` and `self.y`) by the current speed in the `x` and `y` directions.

We'll also call the `draw()` method, which simply calls `blit()` to draw the ball at its current `x`- and `y`-coordinates ❻, in every frame of the main loop.

Using the `Ball` Class

Now all functionality associated with a ball has been placed in the `Ball` class code. All the main program needs to do is create the ball, then call its `update()` and `draw()` methods in every frame. [Listing 6-2](#) shows the greatly simplified code of the main program.